Tech Challenge — Fase 4

Microsserviços e Saga Pattern na Plataforma de Gestão de Oficina Mecânica

> **RASCUNHO PARA REVISÃO DO RAFAEL — NÃO ENVIAR AO PORTAL.** Atualizado em 11/11/2026 com o baseline de `main` da release 3 dos três serviços (SHAs, runs de CI, digests e Sonar; UNR-1207), com o critério de trace decidido pelo Rafael em 10/11 (um `trace.id` por etapa e correlação da OS inteira por `sagaId`; seção 6), com a prova local no Kind desse critério (UNR-1201 e UNR-1204) e com a cronologia das sessões AWS 4 e 5 e das correções posteriores (seção 8.2). Itens marcados **[PENDENTE]**, **[CONFIRMAR]**, `PENDENTE-AWS` ou `PENDENTE-VIDEO` ainda não têm evidência verificável; este texto distingue "implementado/configurado" de "executado e evidenciado" e não declara deploy em nuvem das correções, vídeo gravado, estorno real concluído, trace único da OS inteira (não é o critério adotado), BDD/New Relic nos digests finais no ambiente AWS Academy nem envio da submissão. Prazo de entrega: 17/11/2026.
>
> **Ambiente do vídeo (decidido pelo Rafael em 03/11; ver UNR-613 e UNR-71).** A gravação é feita somente no ambiente AWS Academy. O Kind é apenas validação local/CI antes do deploy e não é ambiente do vídeo. Planos anteriores que tratam Kind como alvo de demonstração estão superados. A prova final de BDD/New Relic/Sonar no ambiente AWS Academy é `PENDENTE-AWS`. O aceite do Kind como evidência de deploy Kubernetes pelos avaliadores **não está confirmado**.

Integrantes: [CONFIRMAR] nome completo e RM de cada integrante (placeholder; dados fornecidos e validados pelo Rafael).

Vídeo de demonstração (até 15 min): `PENDENTE-VIDEO` — URL e timecodes reais só existem após a gravação. Roteiro versionado em `docs/roteiro-video-fase-4.md` (grade de 14min30s, com os caminhos EstoqueInsuficiente e ReparoRecusado).

Repositórios (um por microsserviço) e artefatos finais promovidos em `main`, baseline registrado na UNR-1207 (release 3, 11/11/2026; a consulta a `git`, à API do GitHub e ao Sonar foi feita por aquela tarefa e não foi repetida aqui). Fonte dos digests: esta tabela. O `infra/aws-academy/release-manifest.json` e os overlays `aws` **já refletem** este baseline de `main` de 11/11 em `develop` (SHAs `d9b605bd`, `140dba5d` e `fe2d8844`; digests conforme o `release-manifest.json`; `deliveryRef` `refs/tags/fase4-entrega-2026-11-12`); este documento não os altera.

Serviço	Repositório	`sourceSha` (commit que construiu a imagem, SHA completo)	CI em `main`	Digest da imagem (GHCR)
OS Service (orquestrador da saga)	https://github.com/rafaelxvr/Tech-challenge-15SOAT	d9b605bd445a2442af09cad9cdbeee4f4542f4f9	run 42953101412 success	sha256:704d51aa1f75dc3bce6943768de42d1a8021c69c29fe3b324c09cf2896eb1aa3
Billing Service (orçamento e pagamento)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-billing	140dba5dd782f985ce2d6b5e07acb5aed57114b1	run 42953106505 success	sha256:bb97e1295bc5a33b65839d61f4338c84cdc7f47340aeb59cce0837b6a2368e94
Execution Service (diagnóstico e reparo)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-execution	fe2d8844c6172e3215603d9f97f5da64be22fc3e	run 42953113662 success	sha256:e3832274dc29b5e32174bdffeec4c2e50bd05d481107ec8770d964e16fb3af18

Cada imagem é publicada no GHCR (`ghcr.io/rafaelxvr/oficina-mecanica`, `oficina-billing-service` e `oficina-execution-service`) com tag igual ao `sourceSha`, isto é, o commit que construiu a imagem; neste baseline o `sourceSha` é o commit de merge da promoção `develop` → `main` (core #127, billing #29, execution #54), que incluem o trace por etapa e a anotação de `sagaId` (seção 6). O SHA, o run e o digest de cada serviço são os registrados na UNR-1207; o **Sonar Quality Gate passou nos três** (seção 7). A assinatura/atestado das imagens não foi verificada. **Baseline de aplicação:** os SHAs, digests e runs da tabela acima **substituem** os do pacote de 10/11 (core `e53ecb4`, billing `a0b80f8`, execution `81a8f06`), que servem só de histórico; a sessão AWS 5 foi executada sobre o baseline V11 anterior (seção 8.2). Commits documentais posteriores (manifesto, overlay, Markdown e PDF) **não redefinem as imagens**. **Referência de entrega:** o manifesto traz, por serviço, `deliveryRef` =

`refs/tags/fase4-entrega-2026-11-12`; a tag é criada pelo operador em cada repositório somente depois da promoção documental `develop` → `main`, e é nela (não no `sourceSha`) que se faz o checkout para renderizar e validar os overlays (`verify-overlay.sh --release`). **Até a criação das tags, o `deliveryRef` é uma referência prevista, não verificada.** A tag `fase4-entrega-2026-11-11` é histórica: não foi movida e **não é a entrega final**. Os SHAs de `develop` continuam mudando a cada merge; reconferir antes da submissão. **Limite:** a tabela prova SHA, digest e run de CI; não prova deploy no AWS Academy dos digests novos: as correções (estorno e trace por etapa com `sagaId`) **não foram implantadas nem validadas na AWS**, o que fica para a sessão 6 (UNR-947).

Revisor: [CONFIRMAR] acesso do usuário `soat-architecture` nos três repositórios (na Fase 3 constava nos quatro daquela fase; não verificado para a Fase 4).

Sumário

- 1. Resumo executivo
- 2. Arquitetura dos microsserviços
- 3. Saga orquestrada
- 4. Divisão de responsabilidades e tecnologias
- 5. CI/CD, validação local e evidências de Kind
- 6. Observabilidade (New Relic)
- 7. Qualidade e testes (SonarCloud)
- 8. Pagamento com Mercado Pago (sandbox)
- 9. Matriz de rastreabilidade do enunciado
- 10. Limitações e riscos conhecidos
- 11. Pendências finais

1. Resumo executivo

O desenho da Fase 4 divide a aplicação da oficina em **três microsserviços independentes**, cada um com repositório, pipeline, infraestrutura de execução e banco próprios. Uma **saga orquestrada** coordena as transações que atravessam os três (abertura da OS, orçamento, aprovação, pagamento, reserva de peças e reparo) e registra compensações para falhas recuperáveis.

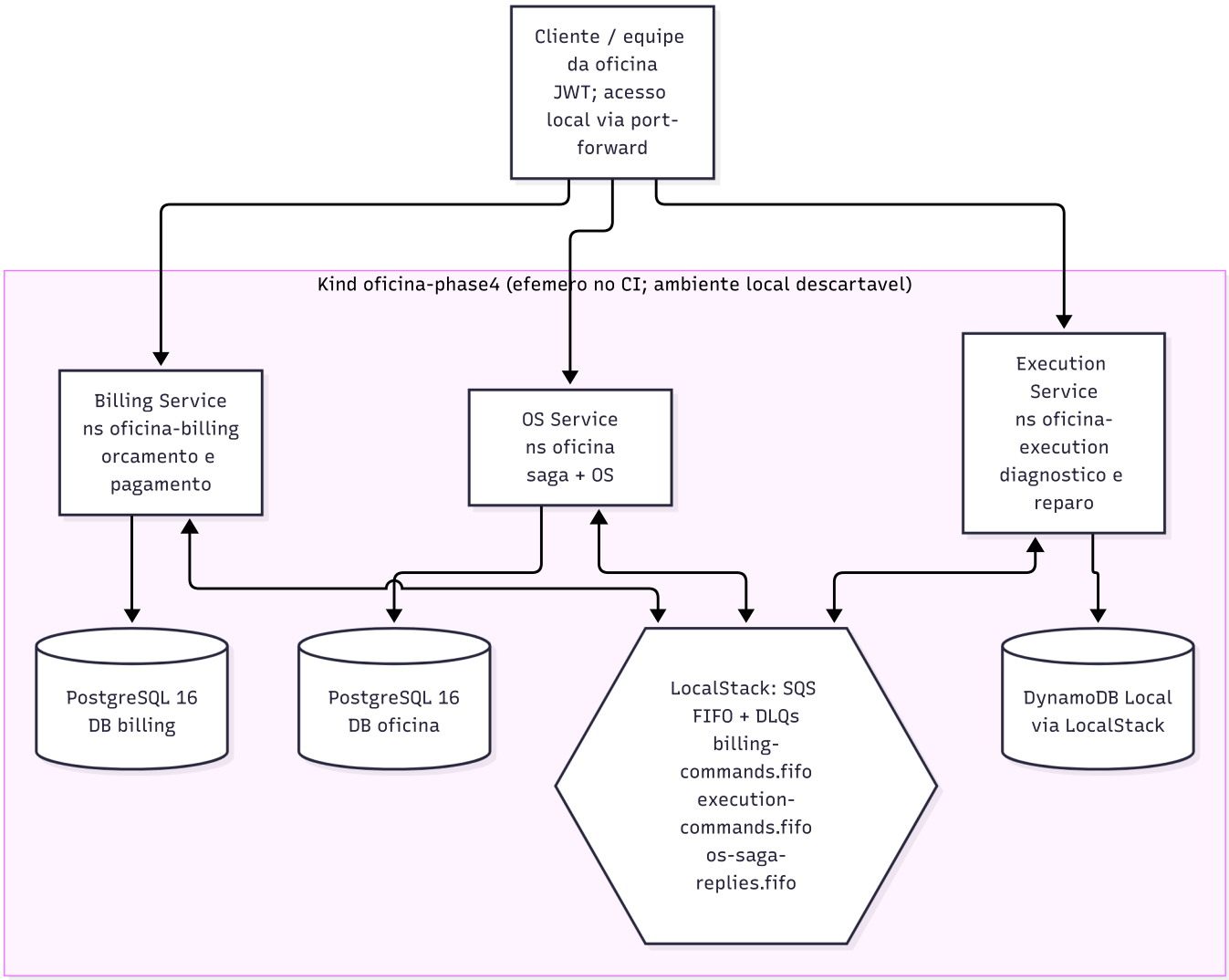
- **OS Service** é dono da ordem de serviço e orquestra a saga. Banco relacional PostgreSQL.
- **Billing Service** é dono do orçamento e do pagamento (Mercado Pago). Banco relacional PostgreSQL próprio.
- **Execution Service** é dono da fila de execução, do diagnóstico e do reparo. Banco não relacional DynamoDB.

Os serviços não acessam os bancos uns dos outros. A integração assíncrona usa filas SQS FIFO exercitadas localmente por LocalStack, com contratos JSON versionados. LocalStack é dependência de teste local; não representa provisionamento ou uso de uma conta AWS.

Situação deste rascunho (11/11/2026): as sessões AWS 4 e 5 já ocorreram (seção 8.2): C1 aprovado, C2 parcial por falha no estorno, C3 pulado, e trace sem propagação pelo SQS; as correções correspondentes (estorno; trace por etapa e `sagaId`) estão em `main` e o trace por etapa foi provado no Kind local (seção 6), mas ainda sem prova na AWS. O Kind local/efêmero é apenas a validação local/CI pré-deploy; o deploy de avaliação e a gravação do vídeo são reservados ao AWS Academy (decisão de 03/11, UNR-613 e UNR-71), com evidência `PENDENTE-AWS`. O estado dos códigos, pipelines, testes, deploys, observabilidade e integração com Mercado Pago deve ser confirmado pelos SHAs e resultados reais antes da submissão; este documento não os declara concluídos.

2. Arquitetura dos microsserviços

2.1 Diagrama geral



Fonte versionada do diagrama: `docs/architecture/phase4-overview.mmd` (SVG renderizado em `docs/architecture/rendered/phase4-overview.svg`).

Diagramas Mermaid versionados: `docs/architecture/c4-*.md` no repositório do OS Service e o diagrama do README do Billing Service. Diagrama geral da validação local Kind, reconciliado com os manifestos: `docs/architecture/phase4-kind-geral.md` (não é evidência de deploy em nuvem).

2.2 Bancos isolados

Serviço	Banco no alvo local	Tipo	Justificativa
OS Service	PostgreSQL 16, banco `oficina`	SQL	OS, histórico e tabelas da saga precisam de atomicidade entre linhas
Billing Service	PostgreSQL 16 próprio, banco `billing`	SQL	Orçamento, pagamento, inbox e outbox precisam compartilhar transações; confere valor e moeda
Execution Service	DynamoDB Local via LocalStack	NoSQL	Fila e estado por OS usam acesso por chave e escritas condicionais atômicas

Nenhum serviço lê ou escreve no banco de outro. Dados de outro domínio só chegam por mensagem.

2.3 Mensageria e contratos

Fila local	Dona (consumidor)	Conteúdo
`billing-commands.fifo` e `billing-commands-dlq.fifo`	Billing	`GerarOrcamento`, `RegistrarDecisao`, `CancelarOrcamento`, `EstornarPagamento`

Fila local	Dona (consumidor)	Conteúdo
`execution-commands.fifo` e `execution-commands-dlq.fifo`	Execution	`IniciarDiagnostico`, `IniciarReparo`, `CancelarExecucao`
`os-saga-replies.fifo` e `os-saga-replies-dlq.fifo`	OS Service	respostas de Billing e Execution

- Envelope comum: `messageId`, `kind`, `type`, `schemaVersion`, `sagaId`, `osId`, `passo`, `causationId`, `occurredAt`, `correlationId`, `traceparent` e `payload`; tamanho máximo de 65.536 bytes.
- `MessageGroupId = osId` e `MessageDeduplicationId = messageId`. Cada resposta copia os identificadores de correlação do comando; `causationId` aponta para o `messageId` respondido.
- Os contratos canônicos são JSON Schemas `contracts/<tipo>/v1.json`: o consumidor é dono do schema de comando e o produtor é dono do schema de resposta. O OS Service fixa cópias de respostas para testes de contrato.
- Inbox idempotente e outbox são persistidos com a transação do caso de uso; as filas FIFO usam DLQ correspondente e `maxReceiveCount = 5`.
- No Kind, LocalStack atende em `http://localstack.infra.svc.cluster.local:4566` com serviços SQS e DynamoDB e credenciais fictícias `test/test`; não há conta nem credenciais reais de AWS.
- REST síncrono: cada serviço expõe sua própria API (`/billing`, `/execucao` e API do OS). Acesso local via `kubectl port-forward`; JWT e permissões são validados nos próprios serviços, sem API Gateway ou ALB. Swagger/OpenAPI e a coleção Postman em `docs/Oficina-Mecanica.postman_collection.json` **[PENDENTES]** de verificação/atualização para a Fase 4 (UNR-605).

3. Saga orquestrada

3.1 Escolha e justificativa

Estratégia: orquestração, com o orquestrador no OS Service (`OsSaga`, tabela `os_saga`). Uma saga por OS começa na mesma transação que abre a OS. Decisão registrada na ADR 006 (`docs/adrs/006-orchestrated-saga.md`).

- Um único lugar guarda estado, etapa atual, prazos e o log de compensação. A falha fica legível, testável e demonstrável (`GET /api/ordens-servico/{id}/saga`).
- O ciclo de vida da OS já vivia no OS Service; o orquestrador fica ao lado do agregado que ele conduz.
- Reaproveita o outbox e o SQS FIFO da Fase 3, sem broker novo.
- Coreografia foi rejeitada: a lógica de compensação se espalharia por três repositórios e ciclos de eventos são fáceis de introduzir. RabbitMQ/Amazon MQ foi rejeitado: infraestrutura nova, sem reaproveitar o outbox, custo extra.
- Custo aceito: o OS Service conhece a ordem das etapas (acoplamento do orquestrador).

3.2 Fluxo feliz

Estados do ciclo da OS definidos no plano: `RECEBIDA → EM_DIAGNOSTICO → AGUARDANDO_APROVACAO → AGUARDANDO_PAGAMENTO → EM_EXECUCAO → FINALIZADA → ENTREGUE`; rejeição do orçamento pode retornar a `EM_DIAGNOSTICO` com nova versão do orçamento. `CANCELADA` é terminal para os caminhos de falha. O estado e a etapa da saga são persistidos separadamente da enumeração da OS.

#	Etapa	Comando → destino	Resposta / avanço
1	Diagnóstico	`IniciarDiagnostico` → Execution	`DiagnosticoConcluido`; o mecânico conclui pela API do serviço
2	Orçamento	`GerarOrcamento` → Billing	`OrcamentoGerado`
3	Decisão do cliente	endpoint existente do OS envia `RegistrarDecisao` → Billing	`OrcamentoAprovado` com `checkoutUrl` ou `OrcamentoRecusado`
4	Pagamento	Mercado Pago ↔ Billing	Billing reconsulta o pagamento verificado e responde `PagamentoConfirmado`
5	Reserva de peças	ação local no OS Service	baixa de estoque com controle otimista; falha por estoque insuficiente inicia compensação
6	Início do reparo	`IniciarReparo` → Execution	`ReparoIniciado` imediato; `ReparoRecusado` se não houver capacidade/filial válida. Este é o pivô da saga
—	Conclusão	Execution conclui pela API do serviço	`ExecucaoFinalizada` encerra a saga e marca a OS `FINALIZADA`

3.3 Compensações

A compensação roda em ordem inversa, a partir de uma pilha persistida. Se o início do reparo for recusado, as ações anteriores podem ser compensadas. `ReparoIniciado` aceito é o pivô; depois dele, a conclusão do reparo não desfaz etapas anteriores.

Falha	Motivo registrado	O que é compensado
Diagnóstico recusado	`DIAGNOSTICO_RECUSADO`	cancelar execução (no-op se nada foi criado) e cancelar a OS
Orçamento falhou ou não foi gerado no prazo	`ORCAMENTO_FALHOU` / `ORCAMENTO_SEM_RESPOSTA`	cancelar orçamento; se ainda não existir, a resposta pode trazer `orcamentoId` nulo; cancelar a OS
Cliente recusou o orçamento	decisão `RECUSADO`	cancelar orçamento; retornar a OS ao diagnóstico e repetir com a próxima versão do orçamento
Decisão expirou	`DECISAO_EXPIRADA` / `DECISAO_SEM_RESPOSTA`	cancelar orçamento e a OS
Pagamento recusado ou expirado	`PAGAMENTO_RECUSADO` / `PAGAMENTO_EXPIRADO`	cancelar orçamento e a OS
Estoque insuficiente na reserva	`ESTOQUE_INSUFICIENTE`	estornar pagamento, cancelar orçamento e a OS
Execution recusou o reparo	`REPARO_RECUSADO` / `REPARO_SEM_RESPOSTA`	repor estoque, estornar pagamento, cancelar orçamento e a OS

Prazos padrão definidos no plano: diagnóstico 3 dias, resposta automática 5 minutos, decisão 7 dias, pagamento 30 minutos e resposta de compensação 5 minutos. A compensação permite até 5 envios; depois disso, `FALHA_COMPENSACAO` exige tratamento pelo runbook. Os fluxos e motivos ainda precisam de evidência de teste antes da submissão.

3.4 Garantias de consistência

- Efeitos (comandos e ações locais), linha da saga e log de passos são gravados na **mesma transação**; o outbox publica depois do commit.
- Respostas são processadas uma vez (`saga_inbox`) e só aceitas quando `causationId` é igual ao comando aguardado; resposta atrasada ou duplicada não tem efeito.
- Toda espera tem prazo; um poller transforma espera vencida em falha e reenvia compensações não confirmadas, até 5 envios no total, antes de `FALHA_COMPENSACAO` (alerta, resolvida por runbook).
- Um pagamento confirmado **depois** de o orçamento ser cancelado é estornado automaticamente pelo Billing, sem resposta tardia para uma saga encerrada.
- `SAGA_ENABLED` e `SAGA_MENSAGERIA_ENABLED` ficam `false` por padrão. Só o perfil BDD/Kind os ativa depois que bancos e filas estiverem saudáveis; com a saga desativada, permanece o fluxo pass-through da Fase 3 e `GET /api/ordens-servico/{id}/saga` responde 404 antes de acessar o repositório.
- O acesso direto no Kind depende da autenticação JWT e das permissões validadas por cada serviço; não há rota de Lambda/API Gateway na validação local vigente.
- OS criadas antes da saga (sem linha em `os_saga`) seguem o fluxo manual da Fase 3.

3.5 Evidência da saga

Evidência	Situação
Fluxo completo e compensações testados com BDD (Kind local)	Executado: run 40884415348 — 34 PASS, 2 cenários cobertos por teste unitário, 0 FAIL, cleanup zerado, incluindo feliz, estoque insuficiente e reparo recusado. **Limite:** pinou SHAs históricos (core `7a8ea6c`, billing `17a37b9`, execution `1bf1e6c`), não os SHAs finais de `main` da tabela da capa. Posterior, no PR #126 (head `54a4e648e5b0016ead78797f4230cf210a8a53fd`, antes da promoção): run 42934949615 `success`, com a matriz BDD e a prova de cleanup verdes (UNR-1204); é run de PR, não de `main`. Reexecução nos candidatos finais no ambiente de avaliação: `PENDENTE-AWS` (UNR-947). A sessão AWS 5 exercitou C1 (caminho feliz) e a parte `EstoqueInsuficiente` de C2 em baseline V11 anterior, não nos SHAs finais (seção 8.2).
Migração de banco	**[PENDENTE]** evidenciar instalação limpa até V11 e fixture V8 atualizada até V11 em run ligado ao SHA atual. O core run 42953101412 (`d9b605b`, `success`) é ligado ao SHA atual; os jobs e logs não foram auditados aqui. A V11 está descrita em 8.1
Smoke e deploy local no Kind	Em `main` final (baseline de 11/11, UNR-1207): core run 42953101412, billing run 42953106505 e execution run 42953113662, todos `success`. Os jobs de Kind desses três runs **não foram auditados nesta atualização** (a verificação de jobs feita em 10/11 vale para os SHAs anteriores). Prova local do trace por etapa no Kind: seção 6

Evidência	Situação
Aceite do Kind como evidência de Kubernetes pelo avaliador	**[CONFIRMAR]** risco externo ainda aberto; não declarar aceite sem confirmação

4. Divisão de responsabilidades e tecnologias

4.1 Por que esta divisão

Serviço	Responsabilidade	Por que separado
OS Service	Abertura, status, histórico e consulta da OS; orquestração da saga	Dono do agregado central e do estado da saga
Billing Service	Orçamento (geração, envio, decisão); pagamento e estorno no Mercado Pago	Integração externa sensível, com segredos e webhook próprios; ritmo de mudança diferente
Execution Service	Fila de execução, diagnóstico e reparo; avisa a conclusão ao OS Service	Carga orientada a fila e acesso por chave, que casa com NoSQL

A divisão segue os contextos delimitados (atendimento, cobrança, execução) citados no enunciado.

4.2 Tecnologias

Item	OS Service	Billing Service	Execution Service
Linguagem e framework	Java 17, Spring Boot 3.2.5	Java 17, Spring Boot 3.2.5	Java 17, Spring Boot 3.2.5
Banco no alvo local	PostgreSQL 16	PostgreSQL 16 próprio	DynamoDB Local via LocalStack
Mensageria no alvo local	SQS FIFO via LocalStack	SQS FIFO via LocalStack	SQS FIFO via LocalStack
Contexto HTTP	API do OS	`/billing`	`/execucao`
Saúde	Actuator	`/billing/actuator/health`	`/execucao/actuator/health`
Imagem e manifestos	Docker, Kubernetes e Kind efêmero	Docker, Kubernetes e Kind efêmero	Docker, Kubernetes e Kind efêmero

[CONFIRMAR] versões exatas contra os `pom.xml` dos SHAs efetivamente incluídos na entrega.

5. CI/CD, validação local e evidências de Kind

5.1 Pipelines

O plano prevê pipeline independente por repositório para build, testes, cobertura, qualidade e deploy de validação em um cluster Kind descartável. O fluxo de branches é `feature` → PR para `develop` → integração → `main` como código entregue. O estado real de cada pipeline e proteção de branch só será afirmado com link e SHA verificáveis.

Item	Situação
Pipeline independente de build e testes nos três repositórios	Em `main` final (11/11/2026), todos `success` e com Sonar Quality Gate aprovado (UNR-1207): core run 42953101412 (SHA `d9b605b`), billing run 42953106505 (`140dba5`) e execution run 42953113662 (`fe2d884`). Não se detalham aqui os jobs de cada run (não auditados nesta atualização); em 10/11, nos SHAs anteriores, SCA, build/testes, SAST, Sonar, imagem e Kind efêmero estavam verdes. SHAs completos e digests na tabela da capa
Proteção de `main` e checagens obrigatórias	Leitura de 03/11: core protegida sem checks obrigatórios/aprovações; billing e execution sem proteção. Mudança de política depende de aprovação do Rafael **[CONFIRMAR]**. Merge continua somente com o Rafael
Cobertura e qualidade no CI	Ver seção 7
Deploy de produção do core	Run histórico (promoção anterior) 41987538475 `SKIPPED` por contrato (`APP_PRODUCTION_DEPLOYMENT_ENABLED` não definida). **Nenhum deploy de produção ocorreu.**
Deploy automatizado de validação	Kind efêmero no CI dos três serviços comprovado nos runs de `main` de 10/11 (SHAs anteriores) e prova local do trace por etapa no Kind com os heads dos PRs #126/#28/#53 (seção 6) — validação pré-deploy. Teardown e health por serviço no ambiente AWS Academy: `PENDENTE-AWS` (UNR-947)

Item	Situação
Imagens e manifesto de release	Imagens publicadas no GHCR e fixadas por digest em `infra/aws-academy/release-manifest.json` (core PR #109, atualizado pelo PR #123 para o baseline de 10/11); digests do baseline de 11/11 na tabela da capa, **já refletidos** no manifesto e nos overlays `aws` em `develop` (frente SRE, UNR-1183; este documento não os altera). O manifesto prova SHA, digest e run de CI; não prova deploy no AWS Academy
Correções da sessão AWS 2	Presentes no histórico de `main` dos SHAs finais (conferido pelos títulos de commits/merges; não são evidência de deploy ou validação com os digests finais): UNR-929, health de mail desligado no perfil `aws` do core (PR #107); UNR-931, Flyway com credencial migradora no perfil `aws` do billing (PR #17); UNR-933, logs JSON com a mensagem real e redação segura nos três serviços (core PR #106, billing PR #16, execution PR #41). Validação em ambiente AWS: `PENDENTE-AWS` (UNR-947)
Correções pós-sessão 5	Em `main` no baseline de 11/11/2026 (promoções core #127, billing #29, execution #54): estorno e erro do provedor no billing (PR #23, UNR-1151); propagação do contexto de trace pelo SQS nos três serviços (core #121, billing #24, execution #49, UNR-1156); e o trace por etapa com `sagaId/correlationId` (core #126, UNR-1190; billing #28, UNR-1199; execution #53, UNR-1200). **Não validadas na AWS** (seção 8.2)
Segurança S-12	UNR-621: core `dd7221e5`, billing `ab95f1bd`, execution `0881f88b`; Gitleaks/SAST/SCA sem bloqueio crítico/alto; riscos residuais documentados na tarefa (SHAs curtos conforme a fonte)

5.2 Validação local pré-deploy no Kind

A validação local pré-deploy usa o cluster Kind `oficina-phase4`, efêmero no CI e descartável no desenvolvimento; ele não é o ambiente de deploy de avaliação nem do vídeo (AWS Academy). A topologia usa namespaces `infra`, `data`, `oficina`, `oficina-billing` e `oficina-execution`; PostgreSQL 16 em serviços separados para OS e Billing; LocalStack para SQS FIFO, DLQs e DynamoDB Local; e imagens carregadas localmente no cluster.

- Nomes de serviço: `oficina-app:8080`, `oficina-billing:8081` e `oficina-execution:8082`.
- Dependências: `postgres-os.data.svc.cluster.local:5432` (`oficina`), `postgres-billing.data.svc.cluster.local:5432` (`billing`) e `localstack.infra.svc.cluster.local:4566` (`sqs,dynamodb`).
- O gate de banco usa uma credencial de migração separada da credencial de runtime: migração limpa até V11 e fixture V8 atualizada até V11. A role de runtime recebe apenas o DML necessário (inicialmente `SELECT/INSERT/UPDATE`); `DELETE` só será concedido se o código provar que é necessário.
- O CI deverá criar o cluster descartável, carregar a imagem, aplicar manifestos, validar prontidão/saúde, guardar a evidência e destruir o cluster.
- Acesso local usa `kubectl port-forward` e validação JWT/roles nos próprios serviços. Não há API Gateway, ALB ou serviço externo de entrada na validação local definida.
- LocalStack usa somente credenciais fictícias em ambiente local. **Não há provisionamento de recursos AWS, deploy em nuvem, Terraform cloud apply nem credenciais AWS nesta validação local.**

[PENDENTE] anexar o resultado real da integração dos três serviços, o SHA de cada imagem, os health checks e a evidência de teardown. O Kind não comprova, por si só, o aceite do requisito de deploy Kubernetes pelo avaliador; esse ponto permanece risco de submissão.

6. Observabilidade (New Relic)

Critério de trace adotado (decisão do Rafael em 10/11/2026). A saga não tem um `trace.id` único para a OS inteira: cada etapa disparada por um **gatilho HTTP** (criação da OS, ações do mecânico, decisão do cliente e simulação/confirmação de pagamento) tem **um `trace.id` próprio**, que atravessa os serviços pelo SQS dentro da etapa. A **correlação da OS inteira** é feita por `sagaId`, anotado como atributo nas transações, com a consulta NRQL `FROM Transaction SELECT ... WHERE sagaId = '<id>'` (por exemplo `SELECT uniqueCount(traceId) ... FACET traceId`; o atributo na conta é `traceId`). **Este documento não afirma trace único da OS inteira.**

Wiring concluído e prova local por etapa. A instrumentação New Relic e a propagação de `traceparent/correlationId` foram mescladas: core #88, #89, #94; billing #8, #11; execution #29, #33; depois a propagação pelo SQS (#121, #24, #49) e o trace por etapa com `sagaId` (#126, #28, #53, todos no baseline de 11/11). **Sessão AWS 5 (antes das correções; NerdGraph/NRQL somente leitura, janela do C1, 10/11):** spans nos três serviços, mas **nenhum `trace.id` atravessou mais de um serviço** (1.475 `trace.id` distintos, todos com um único serviço) e nenhuma transação de fila (detalhes em 8.2). **Prova local no Kind (UNR-1201, parecer QA "aprovado com ressalva"; sem AWS):** com os heads exatos dos PRs (core `54a4e648e5b0016ead78797f4230cf210a8a53fd`, billing `d68ca0640debf761d5e1767d7f1d8cb7cb085f4c`, execution `d9e727b297357065a24d9a72a2f21d766d5b92fa`), agentes Java reais e New Relic real, o cenário BDD-

01a passou e a consulta `WHERE sagaId = <id>` retornou exatamente **4 `traceId` distintos**, um por etapa; as 12 transações com `sagaId` tinham todas `correlationId`; os consumidores de fila tinham `parent.transportType=Queue` e o `traceId` da etapa de origem; os três serviços apareciam. **Limites:** a QA usou um driver em cópia descartável do harness (não versionado) que gerou quatro raízes W3C distintas, uma por etapa; o resultado vale para esses heads, não para os SHAs de `main`; a prova **não** é de trace único da OS inteira. O CI do PR #126 fechou verde (run 42934949615, matriz BDD e cleanup `success`, UNR-1204). A prova na AWS é da sessão 6 (UNR-947). A chave de licença é fornecida fora de banda e nunca versionada.

Item exigido no vídeo	Situação
Sinais `saga_prazo_falhou`, outbox `BLOCKED`, idade do item mais antigo, profundidade de DLQ e falha de compensação	**[PENDENTE]** confirmar instrumentação e anexar execução
Rastreamento distribuído com `traceparent` e `correlationId`	Critério: um `trace.id` por etapa HTTP + correlação da OS por `sagaId` (NRQL). Kind local: 4 traces por `sagaId` provadas (UNR-1201, UNR-1204). AWS: não executado — `PENDENTE-AWS` (sessão 6) + `PENDENTE-VIDEO`
M4: 20 ou mais sagas expiradas distintas sem processar em varreduras consecutivas	**[PENDENTE]** provar limiar e notificação
M7: alerta antes/no limite de cinco envios de compensação ou quando envio bloqueado atingir 20 minutos	**[PENDENTE]** provar limiar e notificação; runbook cobre redrive/reconciliação seguros
Dashboard de saga (volume, duração e falhas)	**[PENDENTE]** anexar dashboard e período observado, se implantado

7. Qualidade e testes (SonarCloud)

SonarCloud foi habilitado nos três repositórios (`SONAR_ENABLED=true`) por core #90 e #92, billing #9 e execution #30, #31.

Medição em `main` (baseline de 11/11/2026). O Sonar Quality Gate **passou nos três serviços** nos SHAs do baseline, conforme registrado na UNR-1207 (gate por serviço; não reconsultado nesta atualização). A **cobertura em código novo** (`new_coverage`, mínimo 80%) dos SHAs de 11/11 está registrada na UNR-1207, com fonte no check SonarCloud do SHA de `main`: core 91,1%, billing 92,6% e execution 93,4%. **Esses valores são de cobertura em código novo, não de cobertura total; a cobertura total de cada repositório não foi medida.** Os valores de 10/11 abaixo ficam só como histórico, por SHA.

Serviço	SHA de `main`	Quality Gate	Cobertura em código novo (`new_coverage`, mín. 80%)	Run
OS (core)	`d9b605bd445a2442af09cad9cdbeee4f4542f4f9`	aprovado (UNR-1207)	91,1% (UNR-1207, check SonarCloud do SHA; código novo, não total) — histórico 10/11, `e53ecb4`: 89,1%	4295310 1412
Billing	`140dba5dd782f985ce2d6b5e07acb5aed57114b1`	aprovado (UNR-1207)	92,6% (UNR-1207, check SonarCloud do SHA; código novo, não total) — histórico 10/11, `a0b80f8`: 92,0%	4295310 6505
Execution	`fe2d8844c6172e3215603d9f97f5da64be22fc3e`	aprovado (UNR-1207)	93,4% (UNR-1207, check SonarCloud do SHA; código novo, não total) — histórico 10/11, `81a8f06`: 93,2%	4295311 3662

Valores históricos de cobertura: em 10/11 (SHAs anteriores), `new_coverage` core 89,1%, billing 92,0% e execution 93,2%; em 09/11, cobertura total de `main` core 88,0%, billing 91,9% e execution 93,6%. Nenhum deles se refere aos SHAs de 11/11, cuja cobertura em código novo está na tabela acima. A UNR-625 mediu branches de PR e a UNR-619 outra métrica (linhas JaCoCo em PR/merge ref e `develop`); não são comparáveis.

Requisito	Situação
Build e testes independentes	Em `main` final (11/11/2026), runs `success` nos três serviços (seção 5.1)
Fluxo completo e compensações com BDD	Kind local: run 40884415348 (SHAs históricos). Reexecução em candidatos finais: `PENDENTE-AWS` (UNR-947)
Cobertura mínima de 80% por serviço	Atendido em código novo: Quality Gate aprovado nos três `main` de 11/11 (UNR-1207), com cobertura em código novo de 91,1% (core), 92,6% (billing) e 93,4% (execution); cobertura total **não medida** (histórico de 10/11 em código novo: 89,1%, 92,0% e 93,2%)
Quality Gate no CI	Quality Gate aprovado nos três `main` de 11/11 (UNR-1207). Sonar e BDD no ambiente AWS Academy: `PENDENTE-AWS` (UNR-947)

8. Pagamento com Mercado Pago (sandbox)

O Billing Service integra o Mercado Pago por uma camada anticorrupção (`GatewayPagamento`): preferência de checkout, consulta de pagamento, busca por `external_reference` e estorno. A notificação do Mercado Pago nunca é confiada: o pagamento é relido no provedor e `external_reference`, valor e moeda (BRL) são conferidos antes de confirmar; divergência não confirma e estorna.

- Webhook `POST /billing/webhooks/mercadopago` com assinatura HMAC, janela de tempo, deduplicação e limites de corpo e taxa.
- Sem túnel HTTPS aprovado, o webhook real não alcança o Kind; a alternativa suportada é a reconsulta (`POST /billing/pagamentos/{id}/reconsulta`), que usa o mesmo caminho de verificação.
- O gateway fake existe só no perfil de testes e **não** é evidência de pagamento real.
- Coleção e environments Postman: PR #85, mesclado em `develop`.
- **[PENDENTE]** Pagamento sandbox real + estorno: ação do Rafael via Postman; `PENDENTE-VIDEO` para a captura. Fake/WireMock não fecha o requisito. Os segredos `MP_ACCESS_TOKEN` e `MP_WEBHOOK_SECRET` são injetados fora de banda e nunca versionados ou registrados em logs.

8.1 Defeito observado na sessão AWS 3: valores em centavos tratados como reais

- **Observado:** na sessão AWS 3 (UNR-1105) o checkout do Mercado Pago cobrou R$ 17.000 onde o esperado era R$ 170 (fator 100).
- **Causa identificada no core (UNR-1110):** o seed `V2__seed_data.sql` gravou preços em centavos (ex.: `12000`) em colunas `NUMERIC(10,2)` cujo contrato é reais decimais, de modo que `12000` valia R$ 12.000,00 em vez de R$ 120,00.
- **Correção entregue:** `V11__corrigir_precos_semeados_para_reais.sql` (core PR #117, baseline `13cd27d`) divide por 100 somente as 10 linhas de serviços e as 10 de peças do seed que ainda coincidem com a impressão digital completa gravada pelo V2; linha editada depois do seed não é tocada e a migration é idempotente. Ordens de serviço já criadas têm snapshots de preço próprios e **não** são alteradas. O contrato de release avançou para V1–V11 (ver `docs/runbooks/first-writer-cutover.md`).
- **Preços do catálogo após a V11 (R$):** serviços: Troca de Óleo 80,00; Alinhamento 120,00; Balanceamento 80,00; Revisão Geral 350,00; Troca de Pastilha 180,00; Troca de Pneu 50,00; Diagnóstico Eletrônico 80,00; Troca de Filtro de Ar 40,00; Troca de Correia Dentada 450,00; Higienização do A/C 150,00. Peças (valor unitário): Óleo Motor 5W30 1L 25,00; Filtro de Óleo Universal 15,00; Pastilha Freio Dianteira 80,00; Pastilha Freio Traseira 70,00; Filtro de Ar Universal 20,00; Correia Dentada + Kit 180,00; Vela de Ignição NGK 35,00; Líquido de Arrefecimento 1L 18,00; Filtro de Combustível 25,00; Amortecedor Dianteiro (par) 180,00.
- **Limite:** a V11 está no core desde `13cd27d` (run de CI `success`) e permanece no baseline de 11/11. A cobrança correta com a V11 aplicada foi observada no C1 da sessão AWS 5 (R$ 170,00, seção 8.2); o checkout sandbox ainda não foi capturado para o vídeo: `PENDENTE-VIDEO`. O billing deste baseline é `140dba5` (inclui as correções da seção 8.2).

8.2 Sessões AWS 4 e 5 (10/11/2026) e correções posteriores

Cronologia, com a evidência de cada etapa; nenhum valor de segredo, identificador de conta ou recurso é reproduzido aqui.

Etapa	Resultado
Sessão AWS 4 (continuação; UNR-1141, resultado em UNR-1138)	**Resultado parcial.** C1 **não concluído**: na reconsulta final o pagamento e o registro no billing estavam `EXPIRADO`, saga e OS `CANCELADA`, sem novo checkout. C2, C3 e trace **não executados**. Destroy concluído (state vazio) e inventário sanitizado zero. Rotação do token sandbox: pendência do Rafael, sem exibir valor
Sessão AWS 5 (baseline V11 anterior; UNR-1145, execução em UNR-1148)	**C1 aprovado:** cobrança de R$ 170,00 (valor correto após a V11), saga `CONCLUIDA`, OS `FINALIZADA`, DLQs vazias e 1 pagamento por OS. **C2 parcial por estorno:** o caminho `EstoqueInsuficiente` funcionou, mas o estorno falhou (`MERCADO_PAGO_ESTORNO_FALHOU`, saga `FALHA_COMPENSACAO`, 0 estornos registrados no provedor); o gateway descartava o erro do Mercado Pago. **C3 pulado** por decisão do Rafael. Infra destruída com inventário zero (UNR-1150). Não foi feita a checagem de passos de saga duplicados (sem evidência)

Etapa	Resultado
Trace New Relic na sessão 5	Consulta somente leitura, janela do C1: spans nos três serviços (≈18,7 mil no OS, ≈8,9 mil no billing e ≈8,7 mil no execution), mas **sem propagação pelo SQS**: 1.475 `trace.id` distintos, todos com um único serviço, todas as transações com `parent.transportType=HTTP`, sem `sagaId/correlationId` como atributo. A saga do C1 fica evidenciada pelo estado da saga e do pagamento, não por trace único
Correção: estorno e erro do provedor (billing #23, UNR-1151)	Em `main` no baseline de 11/11. O erro do provedor deixa de ser descartado e é registrado de forma sanitizada. **A causa original da falha do estorno não foi demonstrada**; será confirmada na sessão 6 pelo log sanitizado do provedor. Nenhuma chamada real ao Mercado Pago foi feita nesta correção
Correção: propagação do trace pelo SQS (core #121, billing #24, execution #49, UNR-1156)	Em `main`. Validação local parcial pela QA (UNR-1167, Kind efêmero próprio e New Relic real, sem AWS, em SHAs de PR anteriores aos finais): dois traces com os três serviços e transações de fila nos dois sentidos, mas com **falha no primeiro salto** (OS iniciado por HTTP → execution abria trace raiz novo). Sem PASS declarado nessa rodada; a falha foi tratada pela correção seguinte
Correção e prova do trace por etapa (core #126, UNR-1190; billing #28, UNR-1199; execution #53, UNR-1200)	Preserva o carrier do trace no primeiro salto HTTP → SQS (cold start) e anota `sagaId/correlationId` (limitados e validados) nas transações. **Decisão do Rafael (10/11):** um `trace.id` por etapa (4 gatilhos HTTP) e correlação da OS por NRQL `WHERE sagaId`; trace único da OS inteira não é o critério. **Prova local no Kind** (UNR-1201, QA aprovado com ressalva; UNR-1204, parecer favorável): 4 `traceId` por `sagaId` e BDD-01a PASS nos heads dos PRs; CI do PR #126 verde (run 42934949615). Não é prova na AWS (seção 6)
Promoções e baseline	Promoções `develop` → `main` feitas pelo operador (core #127, billing #29, execution #54); baseline, runs e digests na capa; Sonar Quality Gate aprovado nos três (UNR-1207, seção 7). O baseline de 10/11 (core #122, billing #25, execution #50) fica como histórico
Sessão AWS 6 (planejada, com o Rafael)	C2 (estorno), C3 e trace por etapa com `sagaId` na AWS, com os digests finais e aprovação do Rafael por recurso. **Pendências honestas:** C2 completo, C3 e a prova de trace na AWS não foram executados

9. Matriz de rastreabilidade do enunciado

Legenda de estado: **Evidenciado** (run/PR e SHA citados); **Parcial** (evidência de SHA anterior ou só configuração); **PENDENTE-AWS**; **PENDENTE-VIDEO**; **Humano** (ação exclusiva do Rafael). Fonte: estado consolidado de 05/11/2026 da auditoria UNR-604. SHAs finais de `main`: tabela da capa.

Requisito	Estado	Evidência verificável (SHA/ambiente)	Lacuna	Dono / próximo gate
Três serviços independentes, persistência isolada (SQL + NoSQL)	Parcial	Repositórios, SHAs de `main` e digests na capa; seção 2.2; smokes de isolamento e grants negativos no run 40884415348 (SHAs históricos, Kind)	Revalidar nos SHAs finais de `main`	`PENDENTE-AWS` — UNR-947
APIs REST documentadas	Parcial	Core `docs/openapi.yaml`; billing `docs/openapi.json`; Springdoc nos três; coleção Postman #85 mesclada em `develop`	Spec do execution em PR #45 (aguarda merge em `develop` do execution); snapshots em `docs/phase4/api/`; coleção validada offline por `python scripts/postman/validate-collection.py` (UNR-961)	TechLead/Backend
Mensageria, contratos, inbox/outbox, Saga	Evidenciado (SHAs históricos)	Seções 2.3 e 3; run 40884415348: 34 PASS, 2 por teste unitário, 0 FAIL	Reexecutar se PRs de instrumentação alteraram envelopes	Rafael/QA
Fluxo feliz ponta a ponta	Evidenciado (Kind, SHAs históricos; AWS, sessão 5)	BDD-01a no mesmo run, cleanup zerado; C1 da sessão AWS 5 aprovado (saga `CONCLUIDA`, OS `FINALIZADA`, R$ 170,00; seção 8.2) em baseline V11 anterior	Reexecução nos digests finais; captura para o vídeo	`PENDENTE-AWS` + `PENDENTE-VIDEO`

Requisito	Estado	Evidência verificável (SHA/ambiente)	Lacuna	Dono / próximo gate
Compensação e idempotência (EstoqueInsuficiente, ReparoRecusado, duplicata, reply tardia, DLQ/redrive)	Parcial (Kind, SHAs históricos; AWS: C2 parcial por estorno)	Matriz BDD do run 40884415348, revisada em UNR-338/339 e aprovada em UNR-69; 06h/07c cobertos por teste unitário (explícito)	Na sessão 5, `EstoqueInsuficiente` OK mas o estorno falhou (`FALHA_COMPENSACAO`); correção em `main` sem prova na AWS; ReparoRecusado e C3 não exercitados na AWS. Reexecução e timecodes no ambiente final	`PENDENTE-AWS` + `PENDENTE-VIDEO` — UNR-947, UNR-613
Mercado Pago sandbox real + estorno	Parcial (pagamento real; estorno falhou)	Pagamento real de R$ 170,00 no C1 da sessão 5; estorno falhou (`MERCADO_PAGO_ESTORNO_FALHOU`, 0 estornos no provedor); correção em billing #23, testada só com doubles; Postman #85	Estorno real concluído e causa original não demonstrados	Rafael; sessão 6; `PENDENTE-VIDEO`
New Relic e rastreamento distribuído	Parcial (critério: um `trace.id` por etapa + `sagaId`; provado só no Kind)	PRs da seção 6; trace por etapa com `sagaId` (core #126, billing #28, execution #53) em `main`; Kind local: 4 `traceId` por `sagaId`, 12 transações com `correlationId` (UNR-1201, UNR-1204); sessão 5 (antes das correções): spans nos três serviços, sem `trace.id` entre serviços	Prova na AWS (sessão 6) e captura no vídeo. Trace único da OS inteira **não** é o critério e não é afirmado	`PENDENTE-AWS` — UNR-947
Cobertura ≥ 80% e SonarCloud	Atendido em código novo (Quality Gate aprovado; cobertura total não medida)	Quality Gate aprovado nos três `main` de 11/11 (UNR-1207); cobertura em código novo (`new_coverage`, check SonarCloud do SHA de `main`): core 91,1%, billing 92,6%, execution 93,4%; runs 42953101412, 42953106505, 42953113662. Histórico de 10/11 em código novo: 89,1%, 92,0% e 93,2%	Cobertura total não medida/afirmada; Sonar no ambiente AWS Academy não executado	`PENDENTE-AWS` — UNR-947
CI/CD, S-12 e política de PR	Parcial	Seções 5.1; PRs para `develop`, merge só pelo Rafael; S-12 em UNR-621	`main` de billing/execution sem proteção; deploy de produção do core `SKIPPED`	Rafael decide política
Kind / Kubernetes automatizado	Parcial	Kind verde em `main` de 10/11 (SHAs anteriores) nos três serviços; baseline de 11/11 com runs `success` (jobs de Kind não auditados aqui); Kind local com o trace por etapa (seção 6) e run 42934949615 verde no PR #126	Aceite do Kind pelos avaliadores **não confirmado**; Kind não é ambiente do vídeo (decisão do Rafael, 03/11)	Rafael (risco de entrega)
Deploy de avaliação AWS Academy	Parcial (sessões 1–5 executadas; correções não implantadas)	UNR-611: sessão 1 aplicada, medida (≈13m30s) e destruída; sessão 3 revelou o defeito de centavos (seção 8.1); sessão 4 parcial e sessão 5 com C1 aprovado e C2 parcial (seção 8.2), ambas destruídas com inventário zero; PR #95 (Terraform/overlay) em `develop`; manifesto de release (core PR #109; já atualizado para o baseline de 11/11 pela frente SRE, UNR-1183)	Os digests de 11/11 (estorno e trace por etapa) não foram implantados na AWS; C2 completo, C3 e trace único dependem da sessão 6 (exige aprovação do Rafael por recurso)	Rafael / UNR-947
Runbook D19-R, DLQ/redrive, reconciliação	Parcial	Harness #91 e runbook #93 em `develop`	Execução online do BDD	`PENDENTE-AWS`
Vídeo ≤ 15 min	PENDENTE-VIDEO	Roteiro: `docs/roteiro-video-fase-4.md` (UNR-613)	Gravação, URL e timecodes	Rafael

Requisito	Estado	Evidência verificável (SHA/ambiente)	Lacuna	Dono / próximo gate
PDF final	Parcial	Este documento e PDF gerados de `docs/entrega-fase-4.md`	Revisão do Rafael, integrantes/RM, URL do vídeo, evidências AWS	Rafael

10. Limitações e riscos conhecidos

- **Prazo:** 17/11/2026. Faltam a sessão AWS 6 (C2 com estorno, C3 e trace por etapa com `sagaId` na AWS, com os digests de 11/11; UNR-947), o estorno real concluído e a gravação do vídeo.
- **Sessões 4 e 5:** a sessão 4 foi parcial e a sessão 5 teve C1 aprovado e C2 parcial (estorno falhou); C3 não foi executado e não havia trace entre serviços. As correções estão em `main`, sem validação na AWS (seção 8.2). **Pendências honestas:** C2 completo, C3 e trace na AWS ficam para a sessão 6.
- **Aceite do Kind:** a aceitação do avaliador como evidência de deploy Kubernetes não está confirmada. O vídeo é gravado só no AWS Academy; Kind é validação local antes do deploy (ver aviso na capa).
- **Evidência de SHA anterior:** BDD e smokes principais foram executados em SHAs históricos; a sessão 5 usou o baseline V11 anterior; a prova do trace por etapa no Kind usou os heads dos PRs #126/#28/#53, não os SHAs de `main`; o ambiente final ainda não repetiu nada disso (CI dos runs de `main` verde, seção 5.1).
- **Deploy de produção:** pulado/desabilitado por contrato; nenhum deploy de produção ocorreu.
- **Webhook Mercado Pago:** sem túnel HTTPS, a confirmação usa reconsulta, não a notificação real.
- **Compensação:** `FALHA_COMPENSACAO` após cinco envios exige tratamento pelo runbook; não é garantia de recuperação automática.
- **Credenciais:** Mercado Pago e New Relic ficam fora de banda; nada neste documento contém segredo.

11. Pendências finais

Este documento **não** afirma que a submissão foi enviada.

1. `PENDENTE-AWS`: sessão AWS 6 com o Rafael (UNR-947): C2 (estorno), C3 e trace por etapa com `sagaId` na AWS, com os digests de 11/11 e aprovação explícita do Rafael por recurso. A tabela da capa cobre SHA, digest e run de CI; não prova deploy.
2. `PENDENTE-VIDEO`: ensaio, gravação, URL e timecodes (UNR-613).
3. Estorno real no Mercado Pago sandbox (Rafael): o pagamento real do C1 foi observado na sessão 5, mas o estorno falhou.
4. Trace: critério decidido pelo Rafael em 10/11 (um `trace.id` por etapa HTTP + correlação por `sagaId`). Provado no Kind (UNR-1201, UNR-1204); falta a prova na AWS (sessão 6). Não afirmar trace único da OS inteira.
5. **[CONFIRMAR]** integrantes e RM; acesso do usuário `soat-architecture` aos três repositórios.
6. **[CONFIRMAR]** política de proteção de `main` (leitura de 03/11: core sem checks obrigatórios; billing e execution sem proteção). A promoção dos três `main` de 11/11 já ocorreu (tabela da capa); as tags `fase4-entrega-2026-11-12` (`deliveryRef`) serão criadas pelo operador após a promoção documental; a tag `fase4-entrega-2026-11-11` é histórica e não é a entrega final.
7. Reconferir SHAs de `develop`/`main`, digests e métricas do Sonar, e regenerar o PDF imediatamente antes do envio.